

Air University



Operating Systems (Lab)

Year 2026

Ms. Qurrat Ul Ain

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: May 22, 2026

Lab # 11

Environment setup:

I navigated to my **Documents** folder, created a new directory named **Lab11** using **mkdir Lab11**, and then entered it using **cd Lab11** to set up my active workspace for the lab task.

```
usama@ubuntu:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  snap  Templates  Videos
usama@ubuntu:~$ cd Documents
usama@ubuntu:~/Documents$ mkdir Lab11
usama@ubuntu:~/Documents$ cd Lab11
```

Task 1 – SJF Without Arrival

i. File creation

I used the **nano sjf.cpp** command inside the **Lab11** directory to open the Nano text editor and create a new C++ source file for the Shortest Job First (SJF) scheduling algorithm implementation.

```
usama@ubuntu:~/Documents/Lab11$ nano sjf.cpp
```

ii. Code

I wrote a C++ program for the non-preemptive Shortest Job First (SJF) scheduling algorithm that sorts processes by burst time, calculates their waiting and turnaround times, and outputs both a formatted results table and a visual Gantt chart timeline.

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    int n;

    cout << "Enter Number of Processes: ";
    cin >> n;

    int bt[n], wt[n], tat[n], p[n];

    // Input Burst Time
    for(int i = 0; i < n; i++)
    {
        p[i] = i + 1;

        cout << "Enter Burst Time for P" << p[i] << ": ";
        cin >> bt[i];
    }

    // Sorting according to Burst Time
    for(int i = 0; i < n - 1; i++)
    {
        for(int j = i + 1; j < n; j++)
        {
            if(bt[i] > bt[j])
            {
                swap(bt[i], bt[j]);
                swap(p[i], p[j]);
            }
        }
    }

    // Calculate Waiting Time
    wt[0] = 0;

    for(int i = 1; i < n; i++)
    {
        wt[i] = wt[i - 1] + bt[i - 1];
    }
}
```

```
// Calculate Turnaround Time
for(int i = 0; i < n; i++)
{
    tat[i] = wt[i] + bt[i];
}

float avgWT = 0, avgTAT = 0;

cout << "\nProcess\tBT\tWT\tTAT\n";

for(int i = 0; i < n; i++)
{
    avgWT += wt[i];
    avgTAT += tat[i];

    cout << "P" << p[i]
        << "\t" << bt[i]
        << "\t" << wt[i]
        << "\t" << tat[i]
        << endl;
}

avgWT /= n;
avgTAT /= n;

cout << "\nAverage Waiting Time = " << avgWT << endl;
cout << "Average Turnaround Time = " << avgTAT << endl;
```

```
// Gantt Chart
cout << "\nGantt Chart:\n";

cout << " ";

for(int i = 0; i < n; i++)
{
    cout << "-----";
}

cout << endl << "|";

for(int i = 0; i < n; i++)
{
    cout << " P" << p[i] << " |";
}

cout << endl << " ";

for(int i = 0; i < n; i++)
{
    cout << "-----";
}

cout << endl;

int time = 0;

cout << "0";

for(int i = 0; i < n; i++)
{
    time += bt[i];
    cout << setw(6) << time;
}

cout << endl;

return 0;
}
```

iii. Compilation & Permission

I compiled the source file using `g++ sjf.cpp -o sjf` and granted execution privileges to the output binary by running the `chmod +x sjf` command.

```
usama@ubuntu:~/Documents/Lab11$ g++ sjf.cpp -o sjf
usama@ubuntu:~/Documents/Lab11$ chmod +x sjf
```

iv. Output

I executed the compiled program with `./sjf`, inputting 4 processes with their respective burst times to successfully generate the sorted metrics table, average timing calculations, and the visual Gantt chart.

```
usama@ubuntu:~/Documents/Lab11$ ./sjf
Enter Number of Processes: 4
Enter Burst Time for P1: 6
Enter Burst Time for P2: 8
Enter Burst Time for P3: 7
Enter Burst Time for P4: 3

Process BT      WT      TAT
P4      3      0      3
P1      6      3      9
P3      7      9     16
P2      8     16     24

Average Waiting Time = 7
Average Turnaround Time = 13

Gantt Chart:
-----
| P4 | P1 | P3 | P2 |
-----
0      3      9     16     24
```

Task 2 – SJF with Arrival

i. File creation

I used the `nano sjf_arrival.cpp` command inside the **Lab11** directory to open the Nano text editor and create a new C++ source file for a Shortest Job First (SJF) scheduling algorithm that incorporates arrival times.

```
usama@ubuntu:~/Documents/Lab11$ nano sjf_arrival.cpp
```

ii. Code

I wrote an advanced C++ program for non-preemptive Shortest Job First (SJF) scheduling with arrival times using a structural data format, implementing a simulation loop that dynamically handles CPU idle time, tracks process states, calculates precise metrics relative to arrival times, and generates a dynamically scaled Gantt chart.

```
#include <iostream>
#include <iomanip>
#include <vector>
using namespace std;

struct Process {
    int pid;
    int at;    // arrival time
    int bt;    // burst time
    int wt;    // waiting time
    int tat;   // turnaround time
    int start; // start time (for Gantt)
    int finish; // finish time
    bool completed;
};

// Function to draw Gantt chart from execution order
void drawGanttChart(vector<pair<int, int>> timeline, int n) {
    cout << "\nGantt Chart:\n";
    // Top border
    cout << " ";
    for (size_t i = 0; i < timeline.size(); i++) {
        int duration = timeline[i].second;
        for (int j = 0; j < duration; j++) cout << "--";
        cout << " ";
    }
    cout << "\n|";
    for (size_t i = 0; i < timeline.size(); i++) {
        int duration = timeline[i].second;
        int pid = timeline[i].first;
        for (int j = 0; j < duration - 1; j++) cout << " ";
        cout << "P" << pid;
        for (int j = 0; j < duration - 1; j++) cout << " ";
        cout << "|";
    }
    cout << "\n ";
    for (size_t i = 0; i < timeline.size(); i++) {
        int duration = timeline[i].second;
        for (int j = 0; j < duration; j++) cout << "--";
        cout << " ";
    }
    cout << "\n";

    // Timeline
    int time = 0;
    cout << "0";
    for (size_t i = 0; i < timeline.size(); i++) {
        time += timeline[i].second;
        cout << "          " << time;
    }
    cout << "\n";
}
```

```
int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;

    Process p[100];
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        cout << "P" << p[i].pid << " - Arrival Time: ";
        cin >> p[i].at;
        cout << "P" << p[i].pid << " - Burst Time: ";
        cin >> p[i].bt;
        p[i].completed = false;
        p[i].wt = p[i].tat = 0;
    }

    int current_time = 0;
    int completed = 0;
    vector<pair<int, int>> timeline; // stores (pid, execution_duration)

    while (completed < n) {
        // Find index of next process to run (SJF among arrived)
        int idx = -1;
        int min_bt = 9999;

        for (int i = 0; i < n; i++) {
            if (!p[i].completed && p[i].at <= current_time) {
                if (p[i].bt < min_bt) {
                    min_bt = p[i].bt;
                    idx = i;
                }
                // if same burst time, choose smaller pid (optional for stability)
                else if (p[i].bt == min_bt && idx != -1 && p[i].pid < p[idx].pid) {
                    idx = i;
                }
            }
        }

        if (idx == -1) {
            // No process has arrived yet -> jump to next arrival time
            int next_arrival = 9999;
            for (int i = 0; i < n; i++) {
                if (!p[i].completed && p[i].at < next_arrival)
                    next_arrival = p[i].at;
            }
            current_time = next_arrival;
            continue;
        }

        // Execute process idx
        p[idx].start = current_time;
        p[idx].finish = current_time + p[idx].bt;
        p[idx].wt = p[idx].start - p[idx].at;
        p[idx].tat = p[idx].finish - p[idx].at;

        // Record for Gantt chart
        timeline.push_back({p[idx].pid, p[idx].bt});

        current_time = p[idx].finish;
        p[idx].completed = true;
        completed++;
    }

    // Calculate averages
    float total_wt = 0, total_tat = 0;
    cout << "\nProcess\tAT\tBT\tWT\tTAT\n";
    for (int i = 0; i < n; i++) {
        cout << "P" << p[i].pid << "\t" << p[i].at << "\t" << p[i].bt
              << "\t" << p[i].wt << "\t" << p[i].tat << "\n";
        total_wt += p[i].wt;
        total_tat += p[i].tat;
    }
    cout << "\nAverage Waiting Time = " << total_wt / n;
    cout << "\nAverage Turnaround Time = " << total_tat / n << "\n";

    // Draw Gantt chart
    drawGanttChart(timeline, n);

    return 0;
}
```

iii. Compilation & Permission

I compiled the source file using `g++ sjf_arrival.cpp -o sjf_arrival` and granted execution permissions to the resulting binary by running the `chmod +x sjf_arrival` command.

```
usama@ubuntu:~/Documents/Lab11$ g++ sjf_arrival.cpp -o sjf_arrival
usama@ubuntu:~/Documents/Lab11$ chmod +x sjf_arrival
```

iv. Output

I executed the compiled program with `./sjf_arrival` and input 4 processes with specific arrival and burst times, generating an accurate performance metrics table, calculating the correct averages, and outputting a dynamic Gantt chart that shows the execution timeline (P1 → P3 → P2 → P4).

```
usama@ubuntu:~/Documents/Lab11$ ./sjf_arrival
Enter number of processes: 4
P1 - Arrival Time: 0
P1 - Burst Time: 7
P2 - Arrival Time: 2
P2 - Burst Time: 4
P3 - Arrival Time: 4
P3 - Burst Time: 1
P4 - Arrival Time: 5
P4 - Burst Time: 4

Process AT      BT      WT      TAT
P1      0        7        0        7
P2      2        4        6       10
P3      4        1        3        4
P4      5        4        7       11

Average Waiting Time = 4
Average Turnaround Time = 8

Gantt Chart:
-----
|      P1      |P3|  P2  |  P4  |
-----
0          7      8      12      16
```